SCHOOL OF NATURAL AND APPLIED SCIENCES

DEPARTMENT OF COMPUTING

TO : MR. S. YUTE

FROM : MALCOM MUNTHALI & WEZZIE MKANDAWIRE

REG # : BSC-COM-NE-10-23 & BSC-COM-NE-22-23

COURSE NAME : NETWORK PROGAMMING AND APPLICATION DEVELOPMENT

COURSE CODE : NET322

TITLE : AsyncIO, HTTP, Serialization, Web Services

DUE DATE : 8th MAY , 2026

YAML is chosen because it is the easiest format for humans to read and write. It uses plain indentation instead of brackets or tags, so it looks clean and natural. Since these files are only read once when the system starts up, performance doesn’t matter.YAML does not enforce a strict schema by default, meaning fields can be freely added or removed without breaking a contract .
Protocol Buffers (Protobuf) on the sensor link. Sensors send readings to the server constantly potentially thousands of messages every second. Protobuf is a binary format, meaning it stores data as compact bytes rather than human-readable text. Protobuf also requires a strict schema, it forces both the sensor and the server to use the same agreed structure, same fields, same data types before any data is even sent. Nobody needs to read these messages directly, so the lack of human-readability is not a problem.
JSON and XML on the public REST API. When apps or dashboards need historical sensor data, they call the public API. JSON is the default because every browser and programming language understands it out of the box. XML is available for older enterprise systems that depend on it. Both are plain text, so developers can read and debug responses easily. The client just tells the server which format it wants, and the server delivers it in that format.

B. A web service protocol is just the agreed set of rules for how two systems send and receive data over a network.
REST gives every piece of data its own web address. If sensor readings are needed, you simply just visit a URL, the same way you visit a webpage. Each request stands on its own, the server does not need to remember the previous request. This makes it easy to handle more users by simply adding more servers. Repeated requests for the same data can also be cached, so the database does not have to do the same work twice.

SOAP wraps every single message in a heavy layer of XML and requires a special document just to explain what the API can do. It was built for large corporate systems and is far too heavy for simply reading sensor data.

While RPC thinks of the API as calling a function like telling the server "run this function and give me the result." It works fine between internal services but does not fit

well with standard web tools, does not support caching easily, and feels unfamiliar to most developers who expect standard web API conventions.

C.The server spends most of its time just waiting for sensor data to arrive, waiting for the database to save it. This is why it's I/O bound. The CPU barely does anything. AsyncIO handles every sensor connection on a single thread. Each connection simply pauses when it has nothing to do and lets others take their turn. One supervisor, the event loop, watches all connections at once and wakes up the right one the moment data arrives. Thousands of idle sensors cost almost no memory at all. While thread costs much of memory just to exist, with thousands of sensors each having it's own thread, wastes a lot of memory and processing power for no reason due to high switching

AsyncIO struggles when the work is heavy calculation rather than waiting, like running a machine learning model. That kind of work keeps the CPU fully busy and blocks everything else. Multiple processes each running on their own CPU core or a threadpool helps.

D. HTTP Mechanics Two HTTP features content negotiation and persistent connections have a direct impact on how this system is designed. In content negotiation**,** the client tells the server what format it wants either JSON or XML using a simple header in the request. The server reads it and responds in that format. This means one API endpoint serves everyone without any extra complexity. If the server cannot deliver the requested format, it sends back a clear error so the client knows what went wrong.
While in persistent connections, every time a client connects to the server, there is a short setup process before any data can flow. Without persistent connections, this setup happens on every single request. For a dashboard checking for new sensor readings every few seconds, that repeated setup adds up. Persistent connections keep the connection open and reuse it, so the setup only happens once. In the AsyncIO server, one coroutine handles one client the entire time receiving requests and sending responses on the same open connection until the client leaves.

REFERENCES

Njema, R., II. (2025, May). *Serialization & web services: JSON, XML, YAML, Protobuf, REST, SOAP, RPC* [Lecture slides]. Department of Computing, School of Natural & Applied Sciences, University of Malawi.

Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Doctoral dissertation, University of California, Irvine]. UCI Libraries.

Python Software Foundation. (2024). *asyncio — Asynchronous I/O*. Python 3.12 documentation